

目录

1. 系统组成

1. 1 硬件

1. 1.1 主机

1. 1.2 调试电缆

1. 1.3 通过 USB 与 PC 连接

1. 1.4 通过 JTAG 与目标连接

1. 1.5 对 PC 硬件的要求

1. 1.6 对目标板硬件的要求

1. 1.7 加电

1. 2 软件

1. 2.1 驱动程序的安装

2. PowerView 调试界面的使用

3. 1 打开调试界面

3. 2 JTAG 连接设置

3. 3 运行脚本文件

3. 4 观察/修改寄存器

3. 5 观察/修改存储器

3. 6 下载程序

3. 7 观察符号表

3. 8 打开程序列表窗口

3. 9 单步执行程序

3. 10 设置软件断点

3. 11 设置 Onchip 硬件断点

3. 12 设置数据观察断点

3. 13 全速运行程序

3. 14 停止运行程序

3. 15 观察变量

3. 16 观察堆栈

3. 17 在线 Flash 编程

1. 系统组成

TRACE-ICP 调试系统由硬件和软件两部分组成，硬件是自行研发的，软件是第三方的。

下面分成硬件和软件两部分来介绍。

1. 1 硬件

TRACE-ICP 的硬件设计采用模块化的结构，分为主机和调试电缆两部分。

1. 1.1 主机

下面三张照片是 TRACE-ICP 主机的顶视图和前视图以及后视图。

图一、TRACE-ICP 顶视图



图二、TRACE-ICP 前视图



图三、TRACE-ICP 后视图

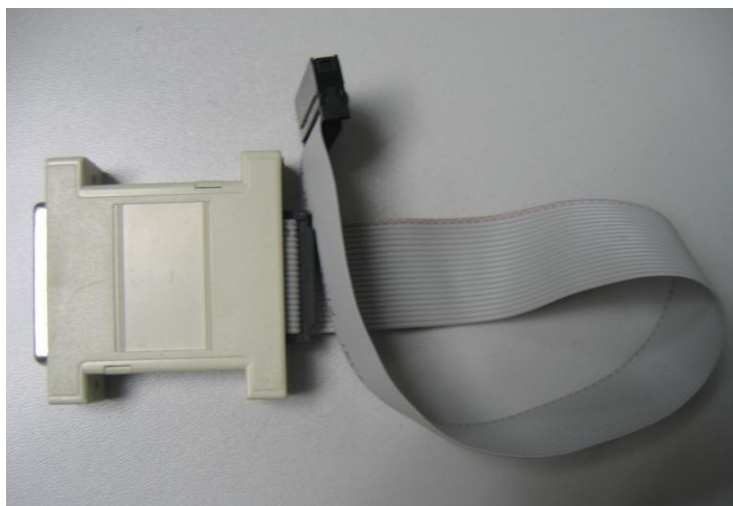


在图二中的连接器是标准 DB25/M 连接器，用于连接调试电缆。在图三中，有两个连接器和一个 LED 指示灯。左边的连接器是 USB 接口，用于通过 USB 电缆和 PC 连接。右边的连接器是 TRACE-ICP 的外接 5VDC 电源接口。TRACE-ICP 可以通过 USB 供电，在 USB 供电不足的情况下，使用外接电源。LED 指示灯是 TRACE-ICP 的电源指示灯。

1. 1.2 调试电缆

下图是 TRACE-ICP 的调试电缆的照片。

图四、TRACE-ICP 的调试电缆



TRACE-ICP 的调试电缆有两个连接端，一个是标准的 DB25/F 连接器，用于和 TRACE-ICP 主机相连，另一个是针距为 2.54 毫米的标准 IDC20 连接器，用于和目标板连接。这个 IDC20 接口通常称为 JTAG 接口，因为这个接口是与目标板上的 ARM 处理器的 JTAG 信号连接的。图中的 20 针扁平电缆可以通过拔插的方式更换。

1. 1.3 通过 USB 与 PC 连接

USB 电缆按照接头连接器的口径来分，有一大一小。大的一端连接 PC 机，小的一端连接 TRACE-ICP 主机。下面的图五和图六说明了这两种连接情况。

图五、USB 线与 PC 连接



图六、USB 线与 TRACE-ICP 连接



1. 1.4 通过 JTAG 与目标连接
- 目标板上的 JTAG 调试接口的信号定义如下图所示。
- 图七、JTAG 调试接口的信号定义

Signal	Pin	Pin	Signal
VTREF	1	2	VSUPPLY (not used)
TRST-	3	4	GND
TDI	5	6	GND
TMS	7	8	GND
TCK	9	10	GND
RTCK	11	12	GND
TDO	13	14	GND
SRST-	15	16	GND
DBGREQ	17	18	GND
DBGACK	19	20	GND

目标板上的 JTAG 调试接口使用针距为 2.54 毫米的 IDC20 连接器。调试电缆上的 IDC20 连接器的上面有一个三角印记，该印记所对应的引脚连接目标板 JTAG 接口的 1 脚。TRACE-ICP 与目标板的连接情况如下图所示。

图八、TRACE-ICP 与目标板的连接



1. 1.5 对 PC 硬件的要求

要求连接 TRACE-ICP 的 PC 能够提供标准的 USB 接口。

1. 1.6 对目标板硬件的要求

目标处理器基于 ARM7/9 内核，处理器 I/O 引脚电源电压为 1.8 到 3.3 伏。

1. 1.7 加电

首先通过 JTAG 接口将 TRACE-ICP 和目标板连接，注意目标板的 JTAG 接口信号要与 TRACE-ICP 的调试电缆的信号一一对应，图七所示是 TRACE-ICP 的调试电缆的 JTAG 接口信号的线序。先给 TRACE-ICP 加电，再给目标板加电。TRACE-ICP 的加电就是通过 USB 线与已上电的 PC 连接。如果 TRACE-ICP 是通过 USB HUB 与 PC 连接，而 USB HUB 的供电能力又不足，可以选择一个接头为内正外负的 5VDC 电源通过 TRACE-ICP 后面的外接电源接口给 TRACE-ICP 供电。

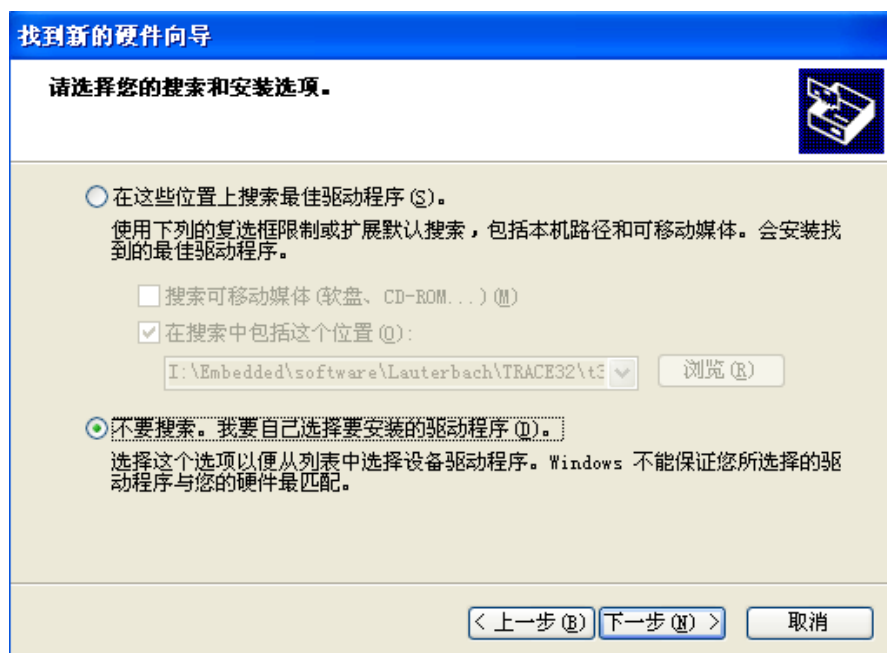
1. 2 软件

TRACE-ICP 能够使用德国 Lauterbach 公司的 PowerView 调试软件，能够使用 GDB 调试软件。

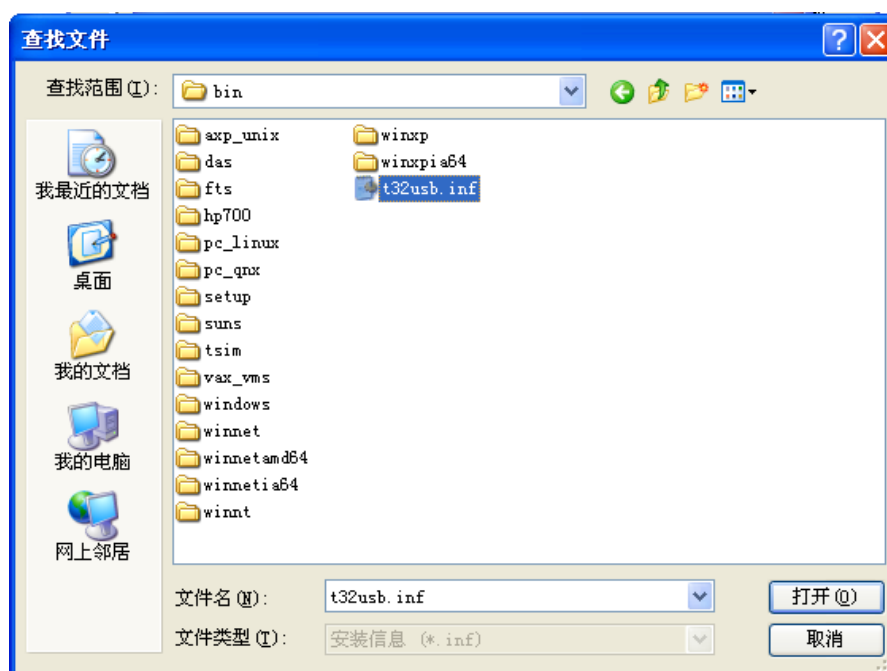
1. 2.1 驱动程序的安装

当第一次将 TRACE-ICP 通过 USB 线与 PC 连接的时候，如果是在 Windows 操作系统下，系统将会提示安装设备驱动程序。驱动程序在安装盘的“bin”目录下，文件名是 t32usb.inf 和 t32usbx.sys。建议客户以手动方式安装驱动程序，速度快。安装步骤如下几幅图所示。

图九、选择手动安装驱动程序



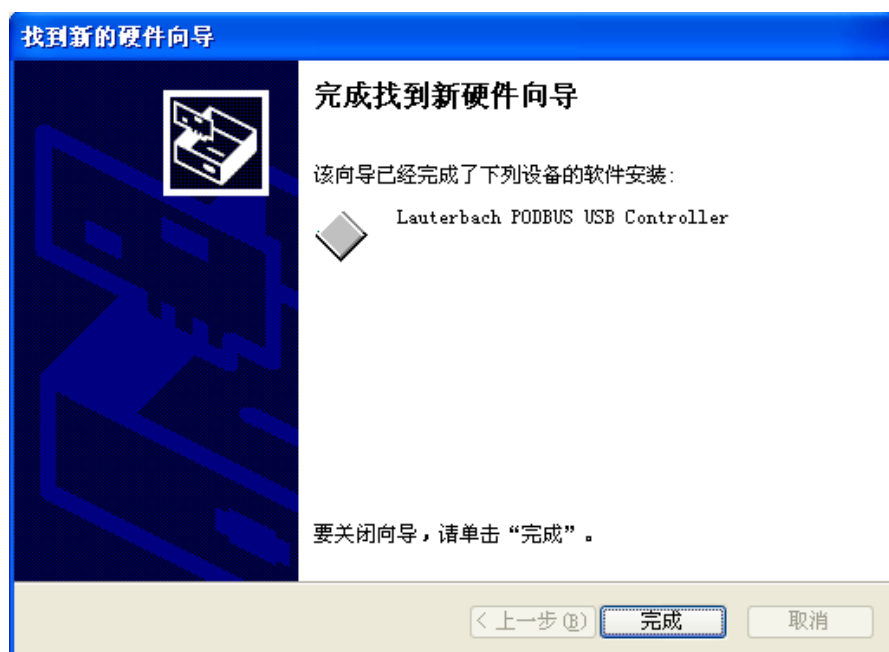
图十、选择驱动程序



图十一、安装驱动程序



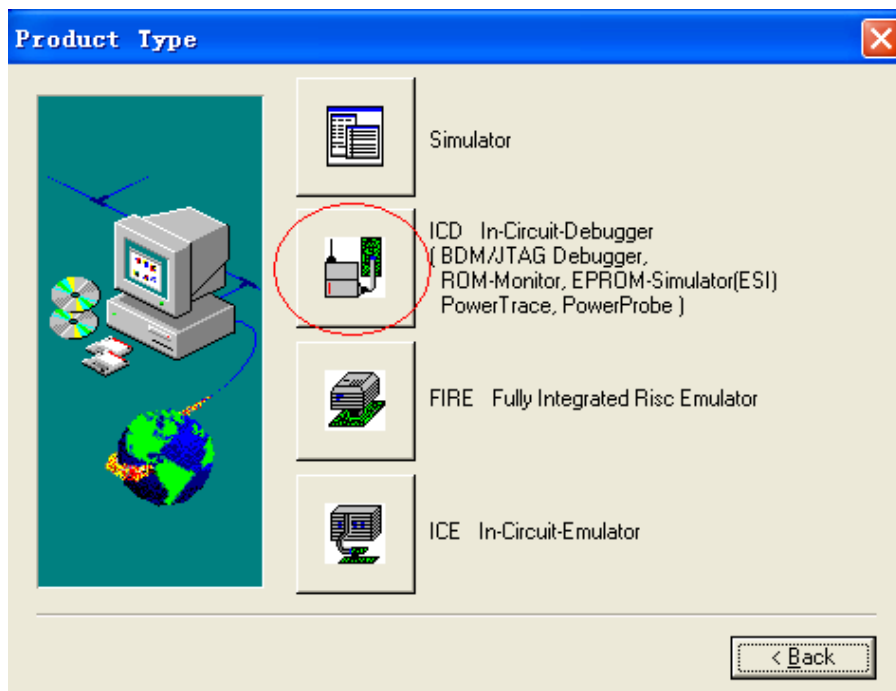
图十二、驱动程序安装完成



1. 2.2 PowerView 调试界面的安装

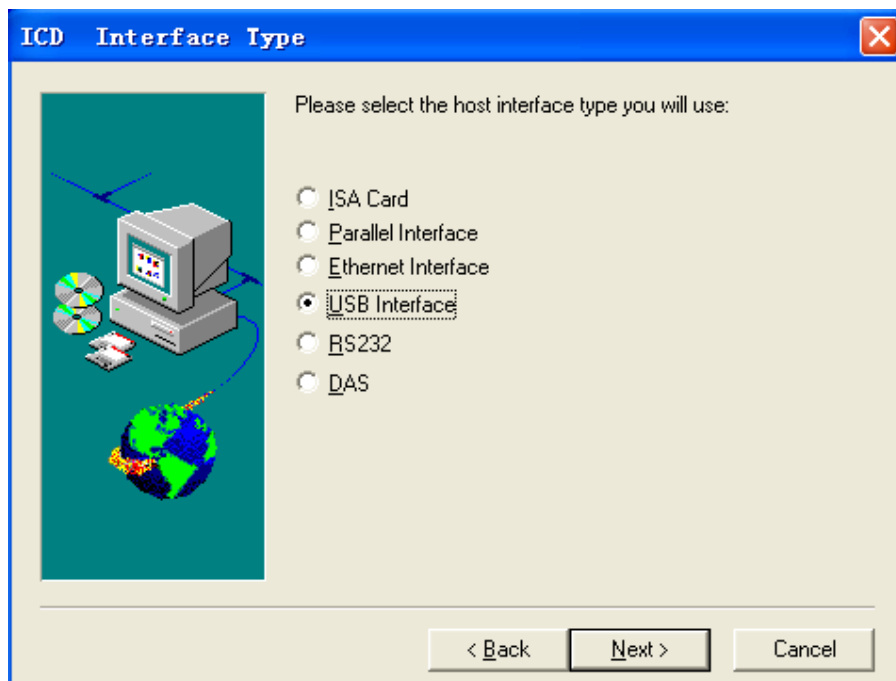
运行安装盘根目录下的 setup.bat 文件。在安装过程中，做下面的几个图示中的选择，就可以正确地安装好调试界面软件。

图十三、选择 ICD 安装



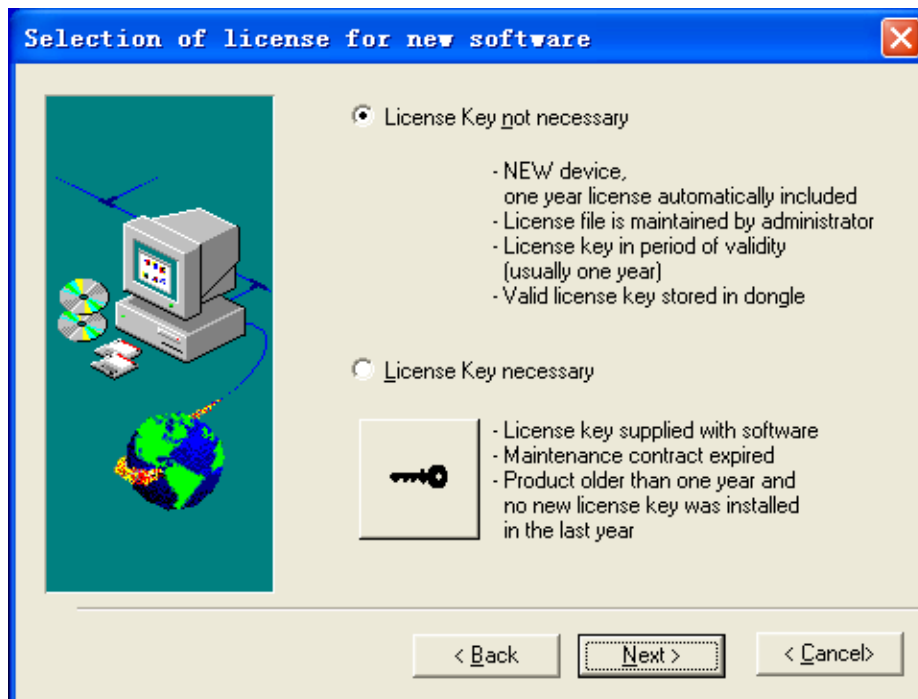
当出现图十三所示的界面时，选择红色圆圈中的按钮。

图十四、选择 USB 接口安装



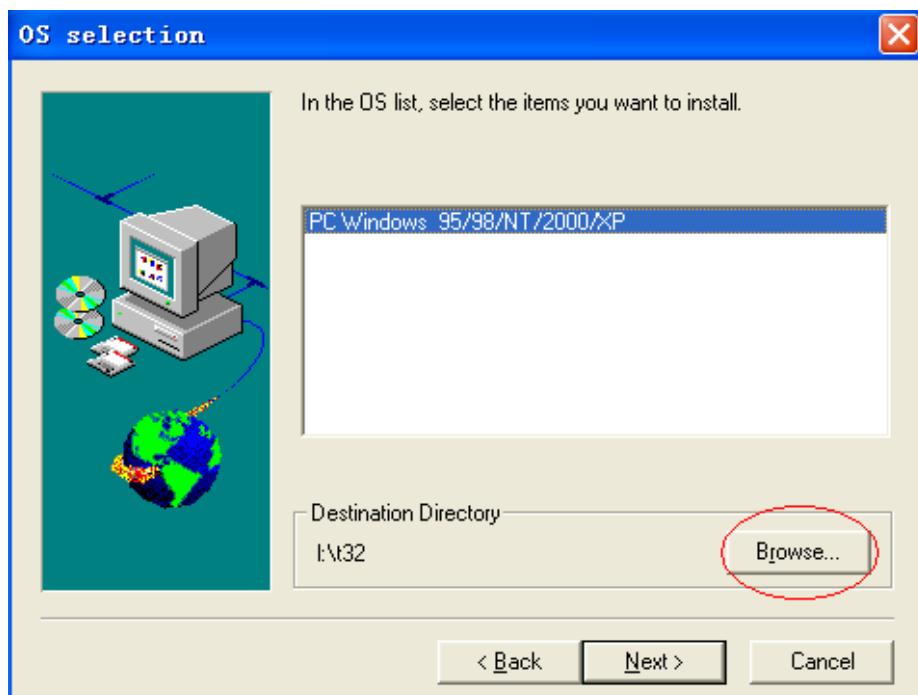
当出现图十四所示的界面时，选择“USB Interface”。

图十五、选择不需要 License 安装



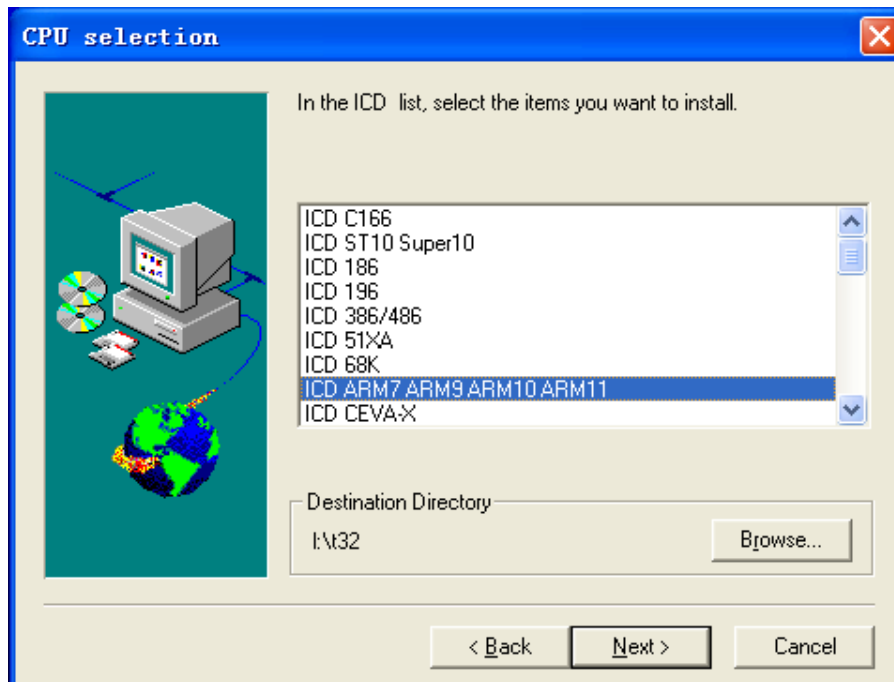
当出现图十五所示的界面时，选择“License Key not necessary”。

图十六、选择安装目录



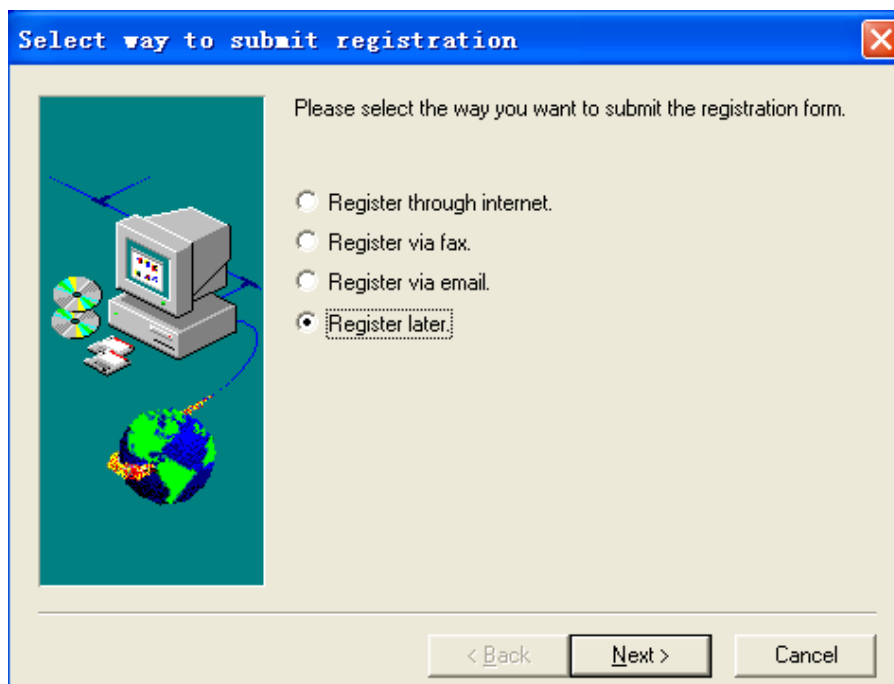
当出现图十六所示的界面时，选择红色圆圈中的按钮，打开安装目录选择对话框，选择将要安装到的目录。

图十七、选择安装的处理器



当出现图十七所示的界面时，选择图中蓝色条所示的处理器列表。

图十八、选择不注册



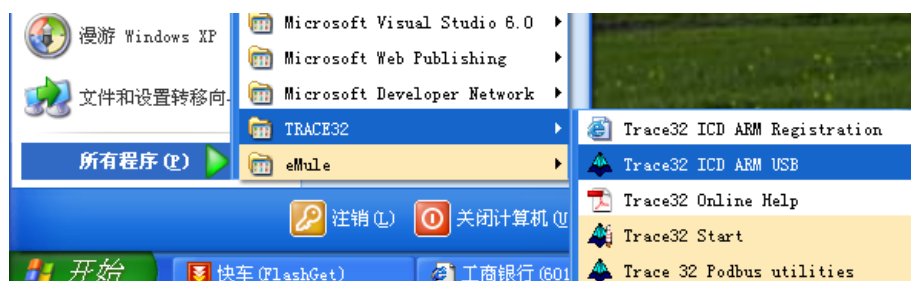
当出现图十八所示的界面时，选择“Register later”，接下来，按照提示完成安装。

3. PowerView 调试界面的使用

3. 1 打开调试界面

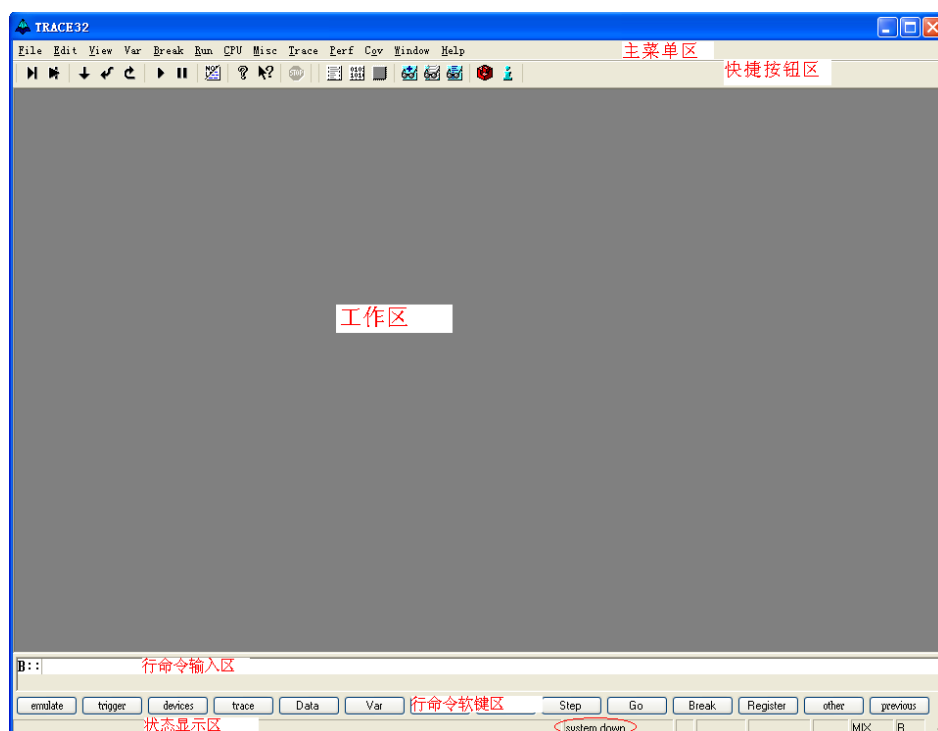
从 Windows 开始—>所有程序—> TRACE32—> Trace32 ICD ARM USB 启动调试界面，如下图所示。

图二十一、启动调试界面



启动之后的调试界面如下图所示。

图二十二、调试界面



在图二十二中的红圈中的“system down”指示目标板已经供电，如果目标板电源电压低或没有的话，红圈的区域会显示“POWER DOWN”。TRACE-ICP 通过 JTAG 接口的 1 脚检测目标板电压，电压范围应该在 1.8 到 3.3 伏之间。如图二十二中红色字体所指示的那样，调试界面分成五个区域，从上到下依次是主菜单区、快捷按钮区、工作区、行命令输入区、行命令软件区、状态显示区。主菜单区是各种菜单命令的入口区域。快捷按钮区是各种常用命令的快捷使用按钮。用户可以自定义主菜单和快捷按钮。工作区是各种对话框窗口的显示区域。行命令输入区是各种命令通过手动输入执行的区域。行命令软键区是

协助用户输入行命令的区域，它提供所有行命令的软键输入方法。状态显示区指示当前的调试状态。

3. 2 JTAG 连接设置

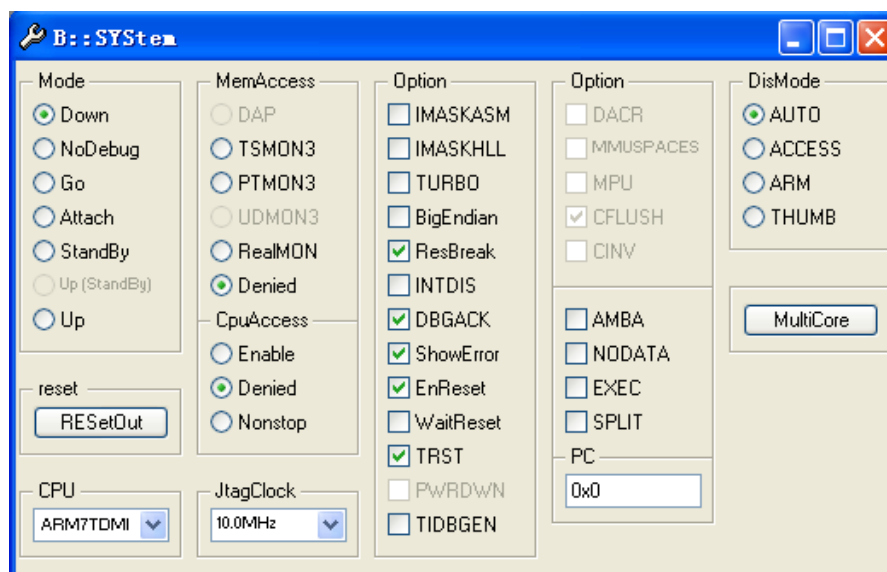
该设置的作用是告诉调试界面目标板 JTAG 链路的设置情况，以便能够正确连接，这些设置主要包括：

- 1、选择要调试的处理器型号。
- 2、是否有多个器件串联在同一个 JTAG 链路里，连接顺序如何，每个器件的 JTAG IR 寄存器的宽度是多少。（情况一）
- 3、JTAG 时钟使用 TCK 还是 RTCK。TCK 由 TRACE-ICP 提供，一般情况下选用 10MHz。RTCK 是 TRACE-ICP 的 TCK 进入目标 JTAG 链路之后，从目标 JTAG 链路返回的时钟，它与目标处理器的时钟同步。一般情况下，具有睡眠模式的处理器多选用 RTCK 作 JTAG 时钟，如 ARM926EJ-S。（情况二）
- 4、通过 JTAG 与目标连接时，是否要先复位目标板。JTAG 口上的 SRST 信号产生复位信号。（情况三）
- 5、通过 JTAG 与目标连接时，是否要停止目标处理器运行。（情况四）

从主菜单“CPU”中选择“System Settings...”，打开如下图所示对话框。

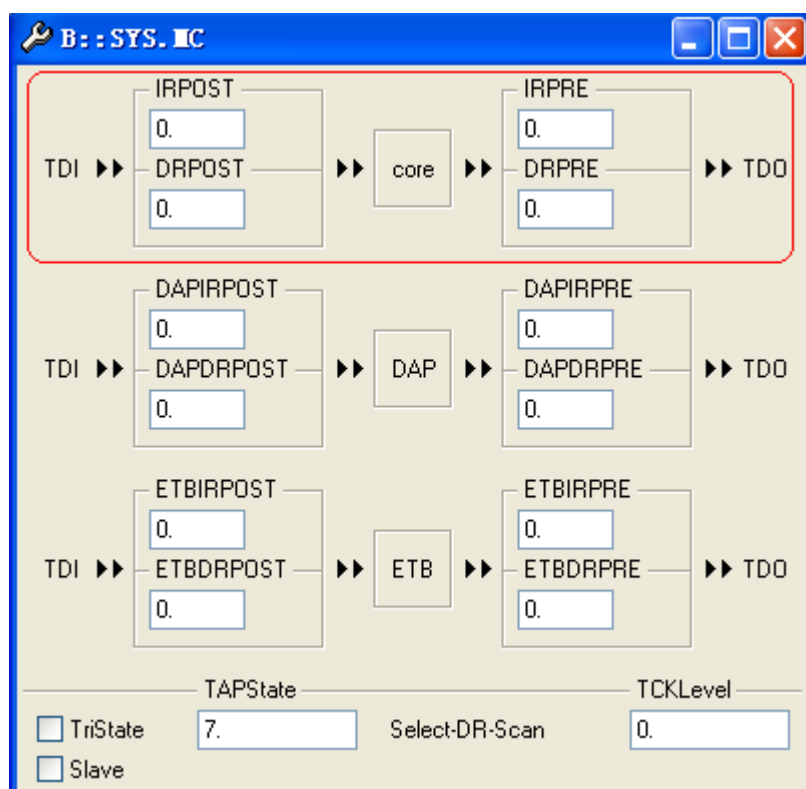
从“CPU”下拉菜单里选择要调试的处理器。

图二十三、System Settings 对话框



对于前面描述的第一种情况，多个器件串联在同一个 JTAG 链上，用户需要在图二十三所示的对话框中选择“MultiCore”，打开 MultiCore 对话窗口，如下图所示。

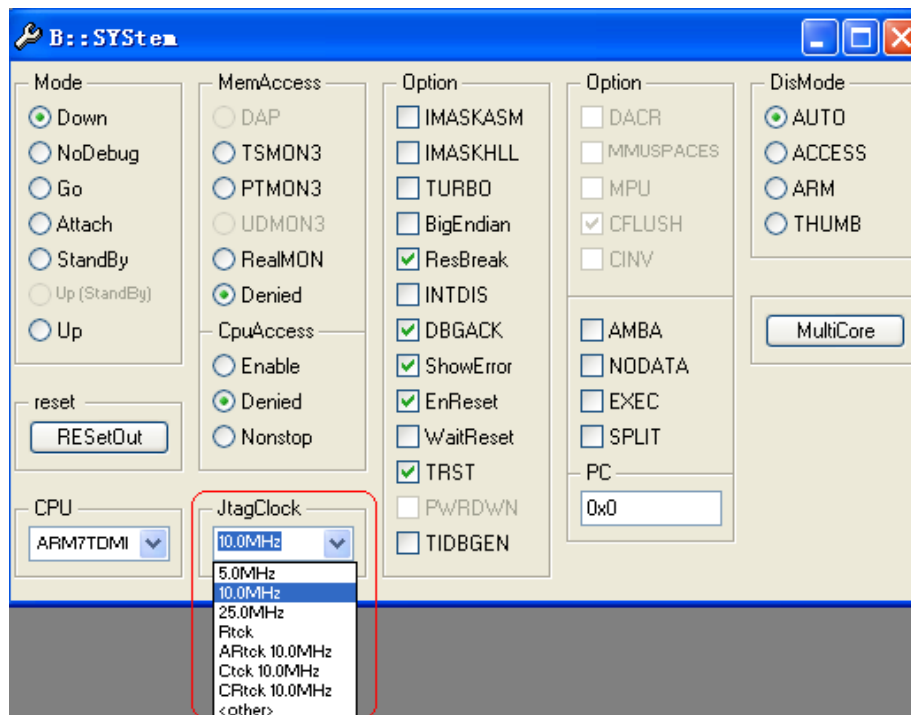
图二十四、MultiCore 对话框



在图二十四中，最上方的红框中的部分描述多个器件在一个 JTAG 链上的位置。所谓“JTAG 串联”，就是一个器件的 TDI 和另一个器件的 TDO 相连，没有连接的 TDI 与 JTAG 口的 TDI 连接，没有连接的 TDO 与 JTAG 口的 TDO 连接。图二十四中的红框中的图形形象地描述了这种连接。在图形中，“core”表示被调试的处理器，如 ARM926EJ-S，“IRPOST”表示连接在 JTAG TDI 和“core”的 TDI 之间的器件的 JTAG IR 寄存器长度的和，在“IRPOST”下方的编辑框内要填入这个和的值，“DRPOST”表示连接在 JTAG TDI 和“core”的 TDI 之间的器件的数目，在“DRPOST”下方的编辑框内填入这个数目值，“IRPRE”表示连接在 JTAG TDO 和“core”的 TDO 之间的器件的 JTAG IR 寄存器长度的和，在“IRPRE”下方的编辑框内要填入这个和的值，“DRPRE”表示连接在 JTAG TDO 和“core”的 TDO 之间的器件的数目，在“DRPRE”下方的编辑框内填入这个数目值。填入上面四个值，就完成了 JTAG MultCore 的设置。

对前面描述的第二种情况，JTAG 时钟的选择，可以通过 System Settings 对话框上的 JtagClock 列表框来实现，如下图所示。

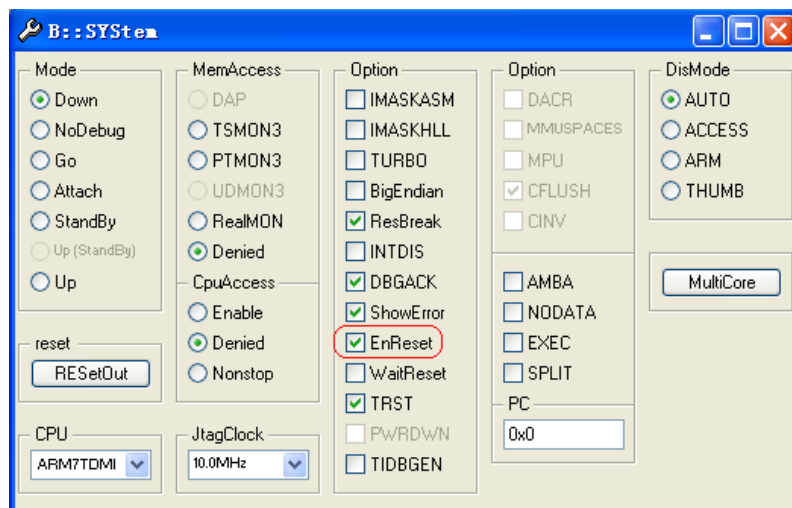
图二十五、JtagClock 列表框



图二十五中的红框中的部分就是 JtagClock 列表框，通过这个列表框用户可以选择 JTAG 时钟是 TCK 或 RTCK，选择 TCK 的时候，顺便选择它的频率，5MHz 或 10MHz 或 25MHz，也可以手动在编辑框中输入频率值，如 1MHz。

对前面描述的第三种情况，通过 JTAG 与目标连接时，是否要先复位目标板，用户可以通过下图中红框中的单选钮进行选择。

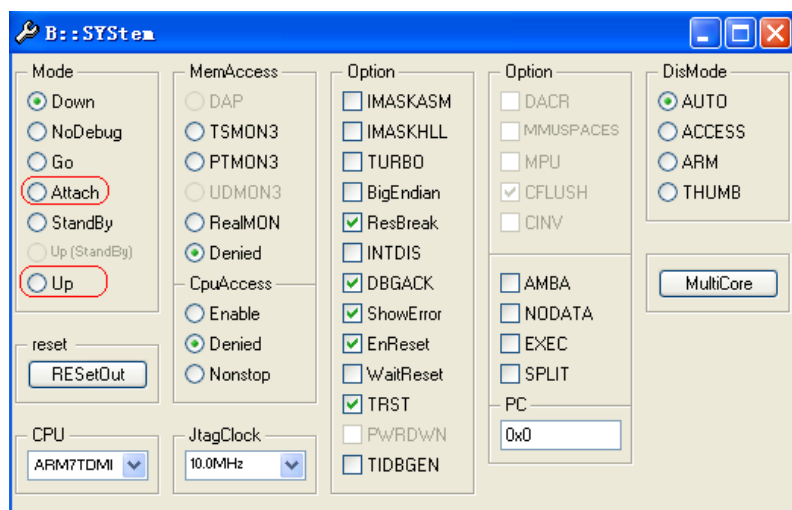
图二十六、系统复位选择



在图二十六中，红框中的“EnReset”单选钮如果在前面打勾（选择），表示在 TRACE-ICP 做 JTAG 连接时会做系统复位。

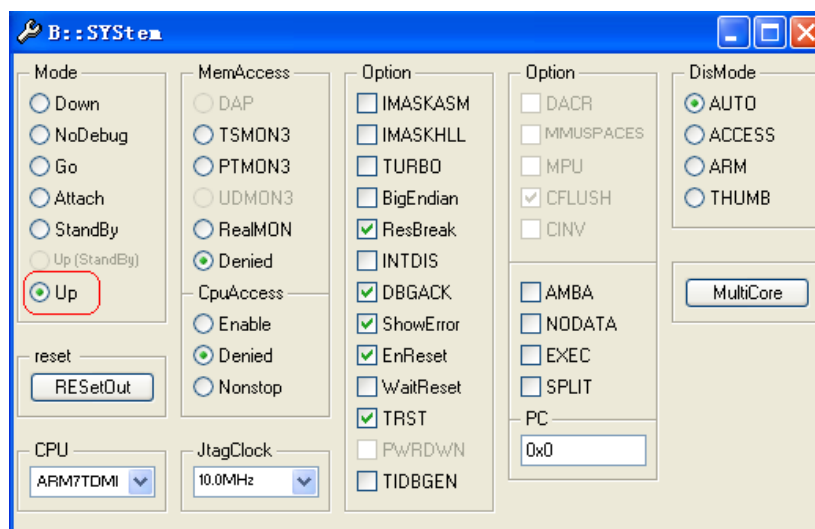
通过前面三种情况，用户完成了在 JTAG 连接动作之前的设置工作，接下来，用户就可以连接目标了。这个连接通过下图中的红框中的“Up”或“Attach”单选钮来完成。

图二十七、JTAG 连接



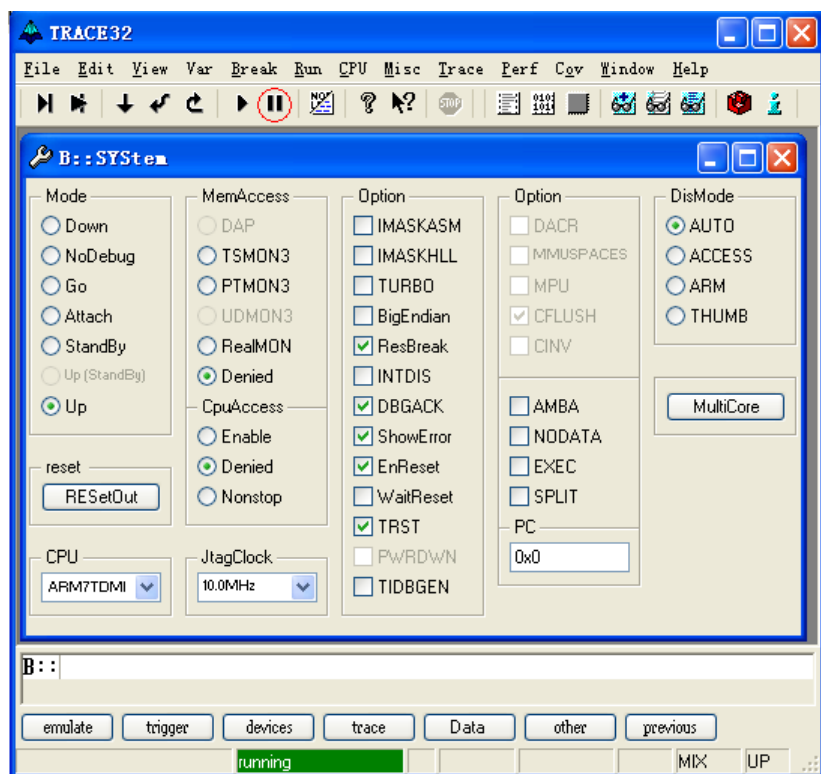
在图二十七中，选择红框中的“Up”单选钮，JTAG 通讯连接之后，目标处理器会停止执行，选择红框中的“Attach”单选钮，JTAG 通讯连接之后，目标处理器处于它在 JTAG 通讯之前的状态，原来是运行的，那么，它现在仍然保持运行状态，这就是我们前面描述的第四种情况，如果用户在选择“Up”或“Attach”单选钮之后，在“Up”前面的小园框中有一个绿色园点，表明 JTAG 通讯已经连接成功。如下图所示。

图二十八、JTAG UP 连接成功



如果选择“Attach”按钮并且目标处理器正在运行的话，在界面的状态显示区会有一个绿色的“Running”条显示，如下图所示。

图二十九、JTAG Attach 连接成功



在图二十九中，用户可以通过点击红圈中的按钮停止程序执行，以便观察程序当前的处理器执行状态。

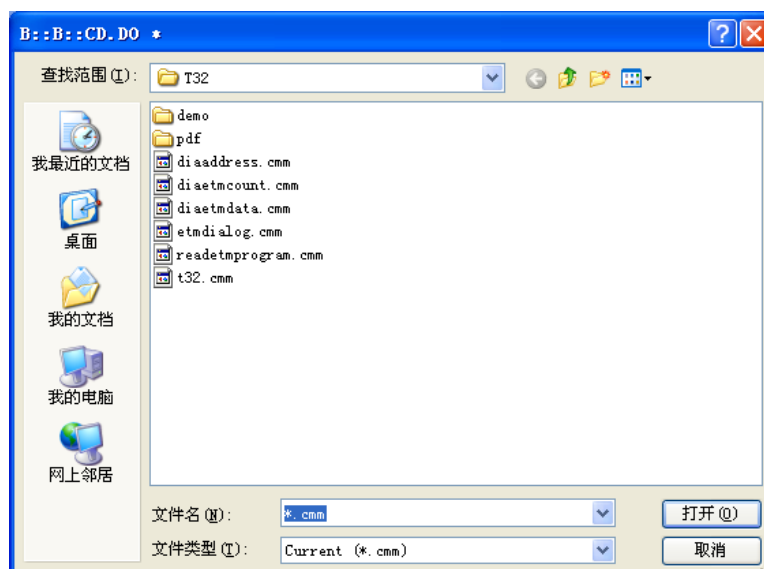
3. 3 运行脚本文件

从主菜单区点击“File->Run Batchfile...”打开脚本文件选择对话框。如下图所示。

图三十、脚本文件执行菜单



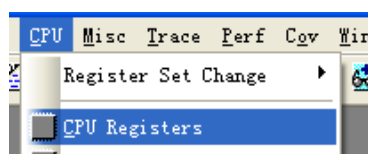
图三十一、脚本文件选择对话框



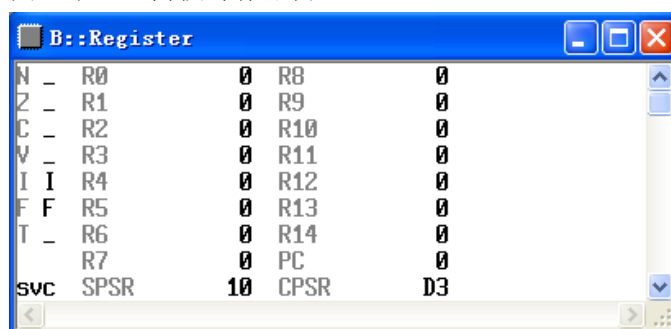
在图三十一所示的对话框中选择要执行的脚本文件，用户可以选择任意目录下的脚本文件。脚本文件的内容主要以调试命令为主。有关脚本文件的编写，请参考软件安装目录的“pdf”目录下的文件“practice_user.pdf”。脚本文件的一般功能是自动执行 JTAG 设置、目标处理器设备寄存器设置、下载要调试的应用程序（支持直接写入 FLASH）、设置调试源文件路径。

3. 4 观察/修改寄存器

从主菜单区点击“CPU->CPU Registers”，打开内核寄存器窗口，如下图所示。图三十二、内核寄存器观察菜单



图三十三、内核寄存器窗口



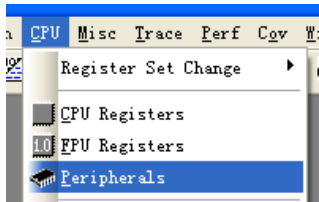
从图三十三所示的内核寄存器窗口，用户能够观察处理器内核寄存器的值。如果用户想修改某一个寄存器的值，只要双击寄存器名右边的值，在行命令输入区就会出现相应寄存器值修改的命令，紧接着输入十六进制的值（如，0x12345678）并回车就可以了。下图是以修改寄存器 R2 的值为例，在行命令输入区出现的命令。

图三十四、修改内核寄存器

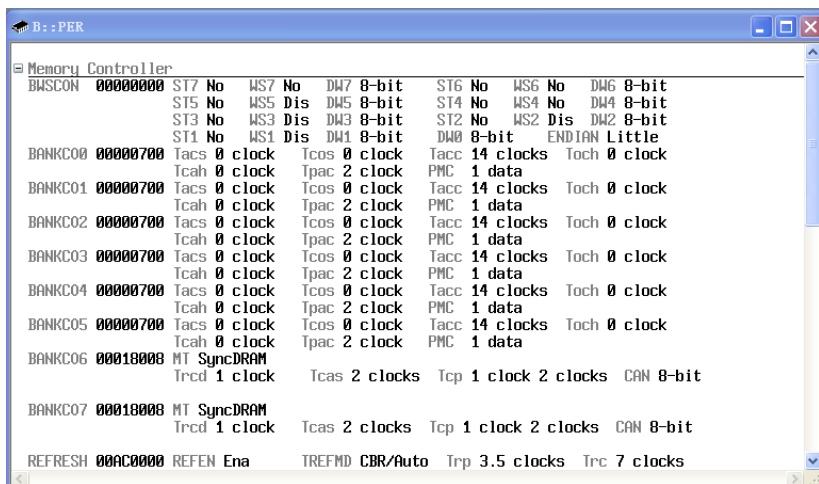


从主菜单区点击“CPU->Peripherals”，打开设备寄存器窗口，如下图所示。

图三十五、设备寄存器观察菜单



图三十六、设备寄存器窗口



图三十六所示的设备寄存器窗口在调试不同的处理器时是不同的。如果用户要修改某个寄存器的值，双击该寄存器的值，在行命令输入区就会出现相应的设备寄存器修改命令，在命令后面输入要修改的值回车即可。如下图所示。

图三十七、设备寄存器修改命令



在图三十七中，设备寄存器的值没有输入。

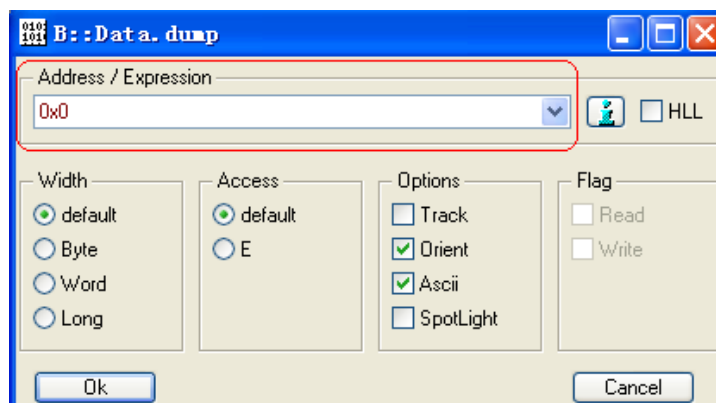
由于设备寄存器映射在处理器的存储器地址空间，所以，也可以用存储器修改的命令修改设备寄存器的值，如 Data.Set。

设备寄存器窗口显示的内容是由一个后缀为“.per”的文件定义的。这个文件是文本的，通过文本编辑器可以编辑，因此，用户可以定制自己的设备寄存器窗口内容。用户在行命令输入区输入“Per.Program”和后缀为“.per”的设备文件，就可以使自己的设备文件有效，设备寄存器窗口就会按这个文件进行显示。

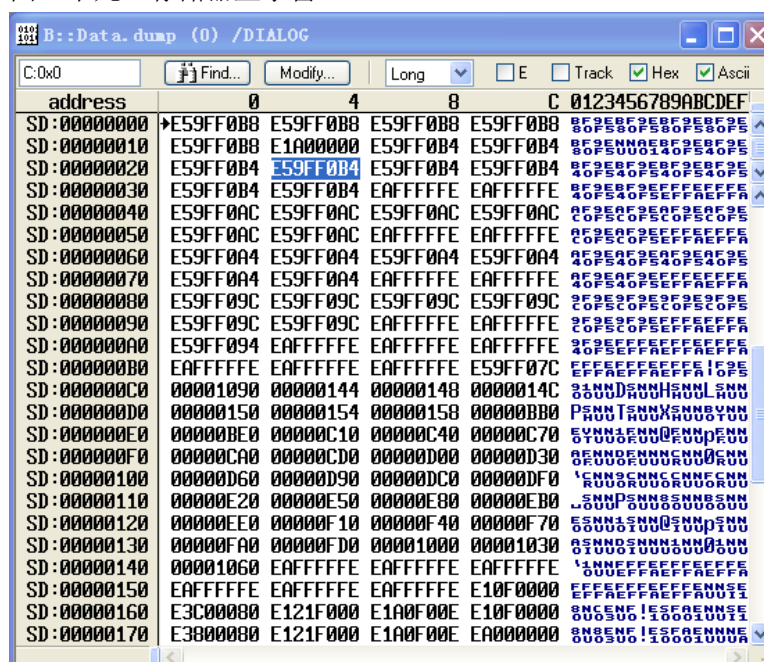
3. 5 观察/修改存储器

从主菜单区点击“View->Dump...”，打开存储器观察窗口，如下图所示。

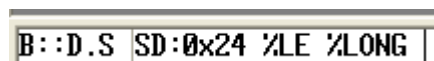
图三十八、存储器地址输入框



在地址输入框中输入要观察的地址，地址也可以用符号方式输入。
输入地址之后点击“OK”按钮，打开存储器显示窗口，如下图所示。
图三十九、存储器显示窗口



用鼠标双击某一个存储单元的内容，在命令行就会出现存储器数据修改命令提示，用户只要填入要修改的数据回车即可。如下图所示。
图四十、存储器修改命令提示



3. 6 下载程序

使用 data.load 命令实现程序下载的功能，如下图所示。

图四十一、下载程序

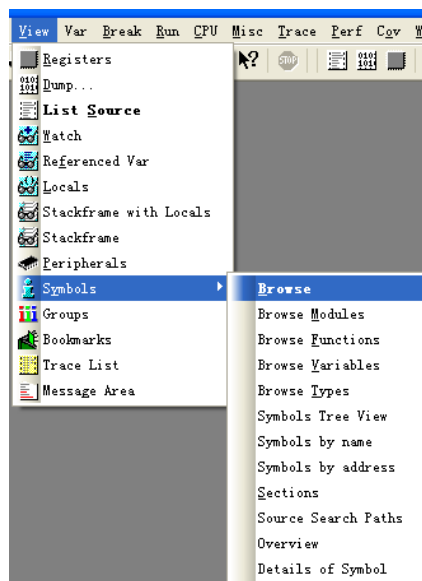


在图四十一中，“elf”指示所下载的程序的文件格式，“/v”指示程序下载完成之后进行校验。

3. 7 观察符号表

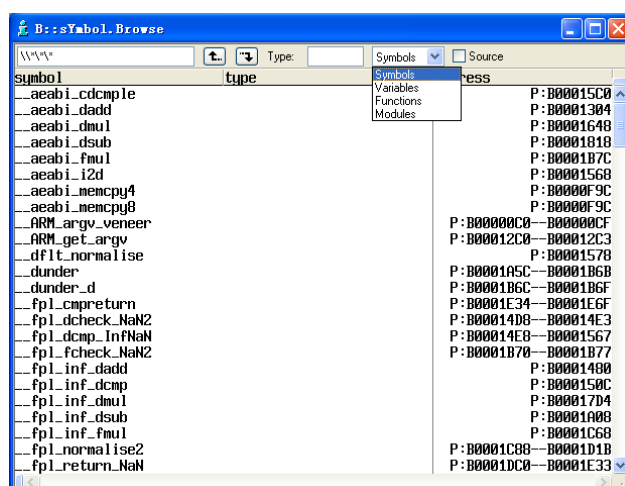
如下图所示，点击“View->Symbols->Browse”打开符号表对话框。

图四十二、打开符号表对话框



符号表对话框如下图所示。

图四十三、符号表对话框

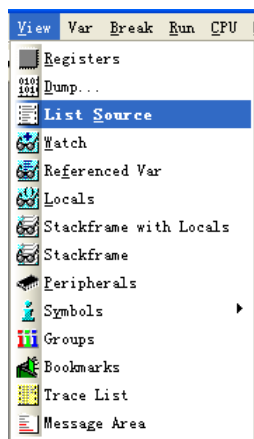


在符号表对话框中可以通过单选钮“Symbols”选择要观察函数或是变量等符号。在符号表对话框中双击变量符号会打开变量观察对话框，双击函数名会打开程序列表窗口。

3. 8 打开程序列表窗口

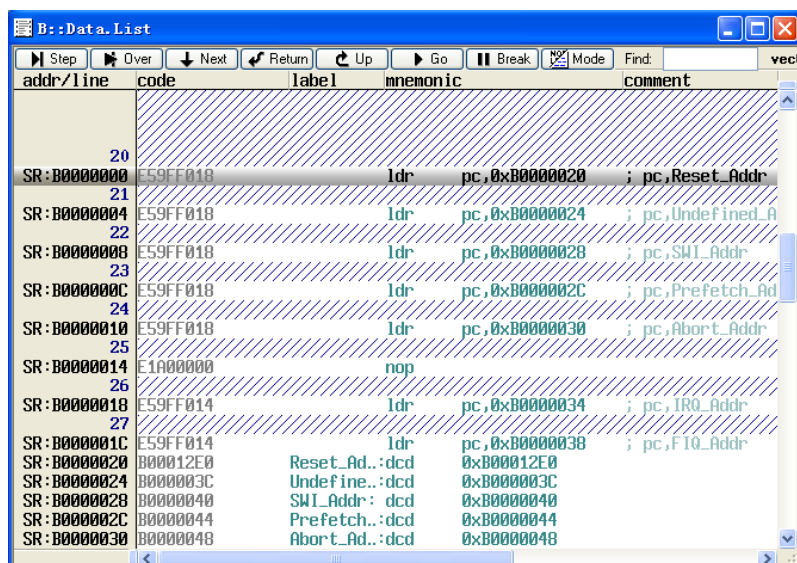
点击“View->List Source”打开程序列表窗口，如下图所示。

图四十四、打开程序列表窗口



打开后的程序列表窗口可以有下面几种形式。

图四十五、找不到源文件的程序列表窗口

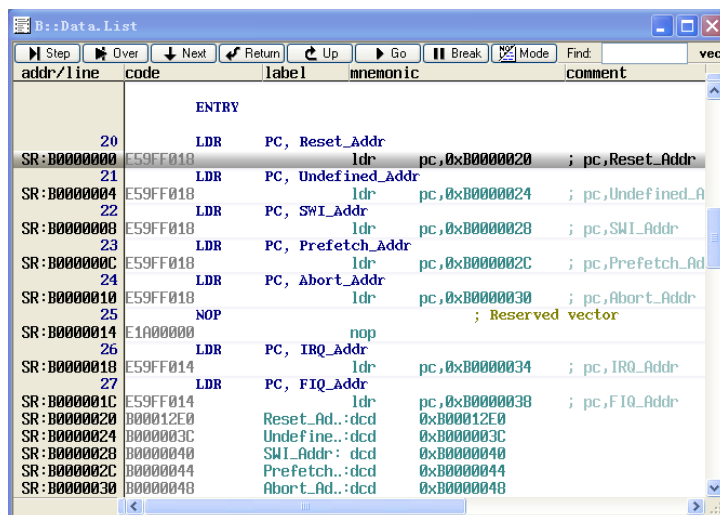


对于图四十五所示的情形，需要用 Y.SPATH 命令指定源程序路径。如下图所示。

图四十六、指定源程序路径

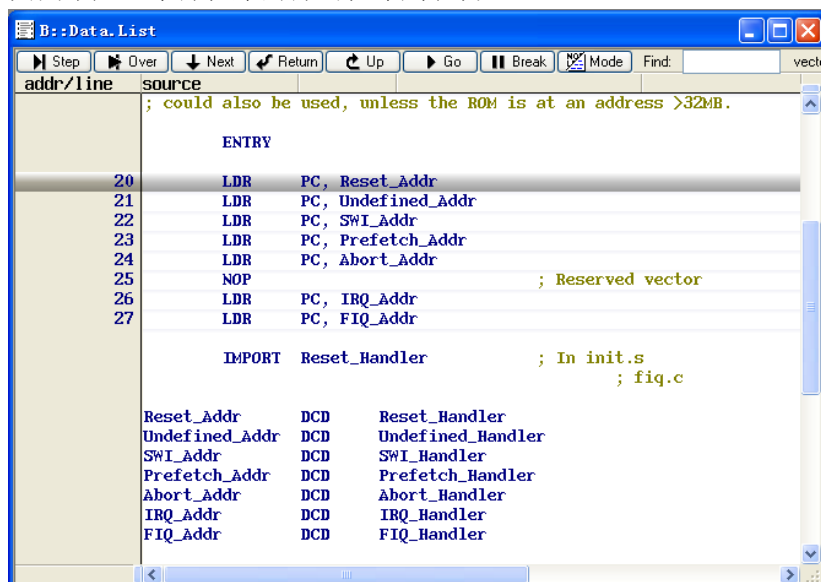


图四十七、带源程序的混合显示程序列表窗口



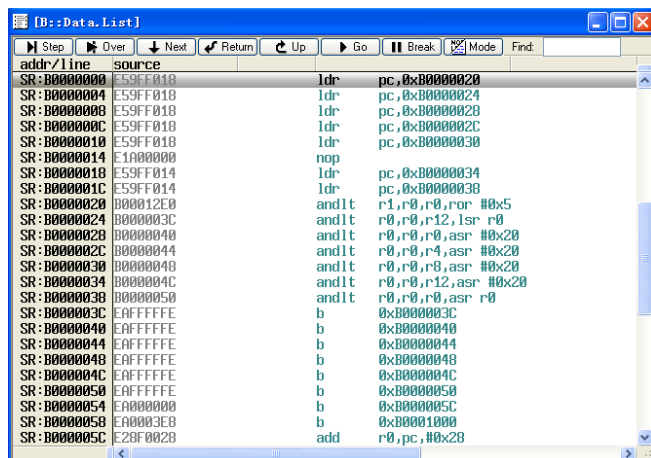
通过点击程序列表窗口上的“Mode”按钮可以切换混合和源码显示方式。

图四十八、带源程序的源码程序列表窗口



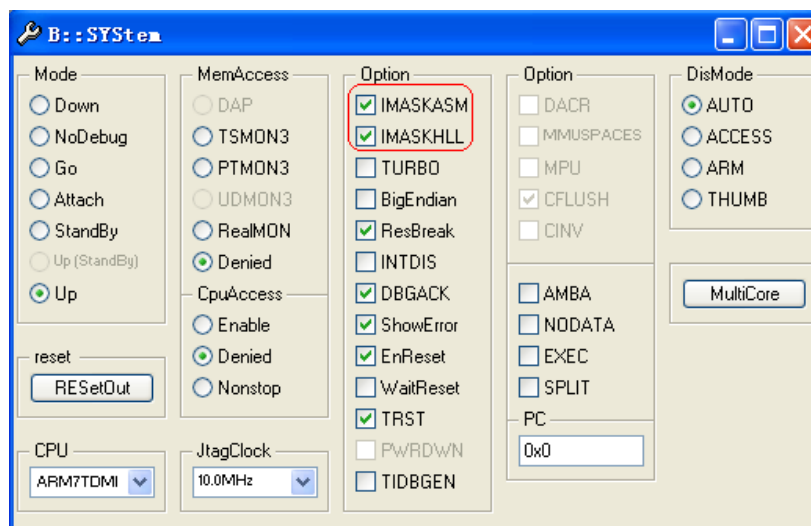
如果用户没有通过 data.load 命令加载符号文件或者所加载的符号文件包含的调试信息不足,用户将会看不到源码,所得到的程序列表窗口可能如下图所示。

图四十九、不带调试信息的程序列表窗口



单步执行程序有 **step** 和 **step over** 两种形式。这两种形式的快捷键分别是 **F2** 和 **F3**。**Step** 的功能是单步执行一条机器指令或高级语言的一行，**step over** 与 **step** 不同的地方在于它可以单步一条函数调用指令或高级语言函数。在混合显示模式，单步以机器指令为单位，在源码模式下，单步以源码程序行为单位。单步执行程序时可以屏蔽中断，如下图所示。

图五十、屏蔽中断



在图五十中红框内的单选钮 **IMASKASM** 和 **IMASKHLL** 选择之后，单步时就会屏蔽中断。用户也可以通过命令 `sys.o imaskasm on` 和 `sys.o imaskhll on` 来设置这两个选项。

3. 10 设置软件断点

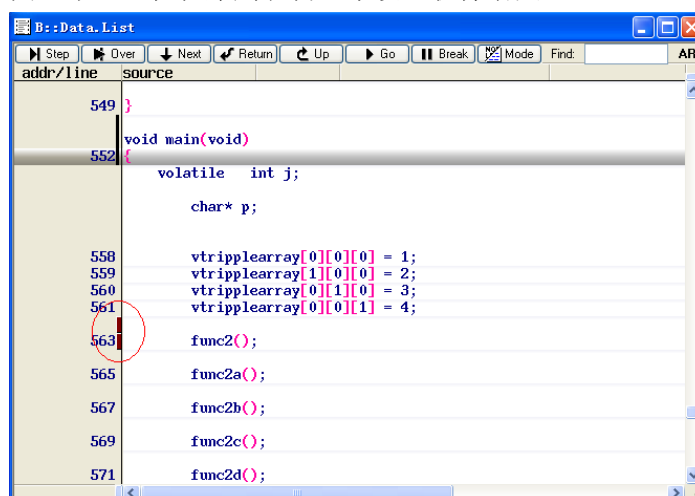
设置软件断点可以在命令行输入命令 `break.set <address> /soft` 来实现，在命令中的 `<address>` 代表程序地址，可以是程序中的函数名等符号。如下图所示。

图五十一、用命令设置软件断点

```
B::break.set 0x0 /SOFT
```

也可以通过在程序列表窗口的程序指令或源码旁边的空白处双击鼠标左键，直接在看到的程序上设置软件断点。如下图所示。

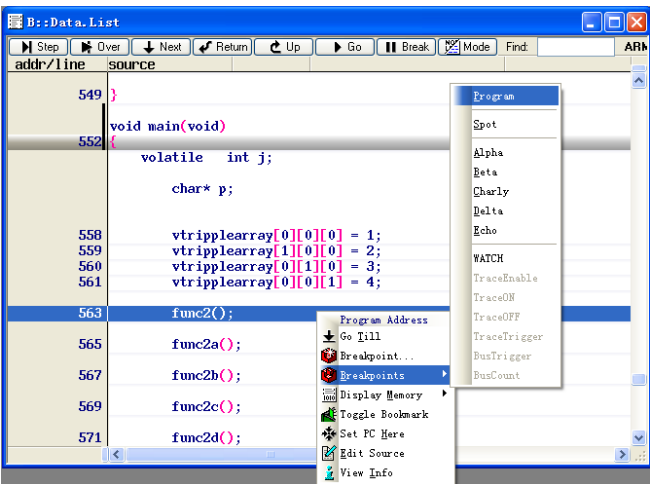
图五十二、在程序列表窗口中设置软件断点



在图五十二中，红色圆圈中的标示就是断点标示。另外，用户还可以在程序列

表窗口中点击鼠标右键，打开辅助对话框，选择 Breakpoints->Program。如下图所示。

图五十三、通过鼠标右键设置软件断点



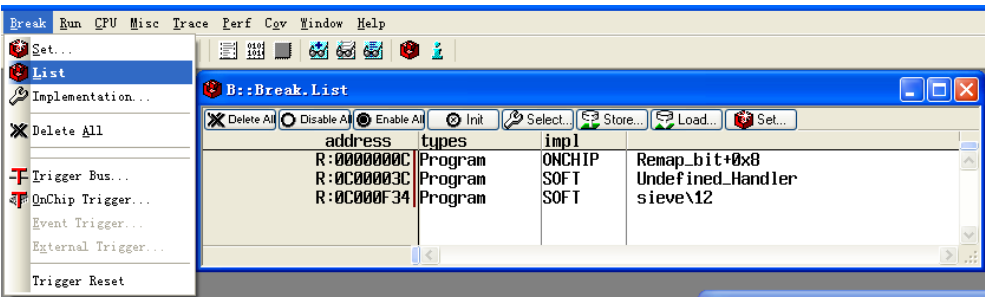
如果在设置软件断点之前执行了 map.bonchip <range>命令，并且所设置的软件断点在<range>所指的地址范围内，那么，通过双击鼠标左键和单击鼠标右键设置软件断点的方法所设置的断点将是 onchip 硬件断点。如果用户在 CPU 不能进行正确写操作的地址上设置软件断点，将会出现下图所示的错误提示。

图五十四、软件断点错误提示



如果用户要察看所有的断点，可以从主菜单点击 Break->List 打开断点列表。如下图所示。

图五十四、断点列表



在断点列表上，用户可以用鼠标左键双击某一断点，打开该断点所在的程序列表窗口，用户也可以用鼠标右肩单击某一断点，激活断点列表窗口的右键辅助功能，对该断点做使能/除能、删除、改变设置等操作。

还可以从主菜单选择 “Break->Set” 打开断点设置对话框，如图五十六。在 “address/expression” 中输入断点的地址或者符号（点击叹号打开符号表从中选择），然后选择 “implementation” 下的 “SOFT”，最后，点击 “set” 完成设置。

3. 11 设置 Onchip 硬件断点

设置 Onchip 硬件断点有几种方式。

一种是通过在命令行输入命令的方式。命令格式为：

break.set <address> /onchip

<address>可以是源码符号。

如下图所示。

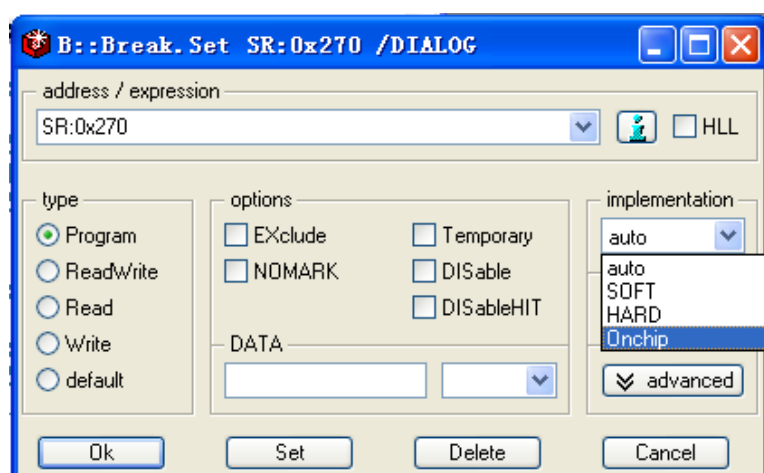
图五十五、通过命令行设置 onchip 硬件断点

```
B::break.set c:0x268 /onchip
```

一种方法是通过先执行 map.bonchip <range>, <range>表示地址范围, 然后, 再在属于这一范围的程序列表窗口中的指令或源码程序行上双击鼠标左键, 来设置 onchip 硬件断点。

一种方法是在程序列表窗口中的指令或源码程序行上单击鼠标右键, 打开右键辅助功能, 选择 “breakpoints...”, 打开断点设置对话框, 从 “implementation” 下拉选项中选择 “Onchip”, 然后, 单击 “Set” 按钮, 完成设置。如下图所示。

图五十六、通过断点设置对话框设置 onchip 断点



一种方法是直接从主菜单中选择 “Break->Set” 打开断点设置对话框, 如图五十六。在 “address/expression” 中输入断点的地址或者符号 (点击叹号打开符号表从中选择), 然后选择 “implementation” 下的 “Onchip”, 最后, 点击 “set” 完成设置。

3. 12 设置数据观察断点

数据观察点的作用就是当程序对这个观察点所在的地址或地址范围进行读或写操作时, 能够停止程序的执行。

设置数据观察断点时, 可以指定数据的内容 (可以是位屏蔽的) 和类型 (字节、半字、字), 也可以不设定。如果设定了, 那么, 只有在这些数据内容匹配的时候, 程序才会停下来。

设置数据观察断点的命令格式如下:

```
break.set <address|range> /readwrite|read|write {data.byte|word|long <value>}
```

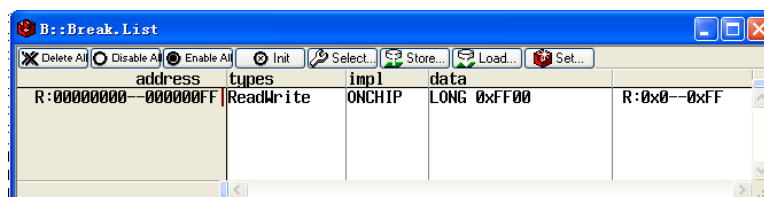
如下图所示。

图五十七、通过命令行设置数据观察断点

```
B::b.s 0-0ff /READWRITE /DATA.LONG 0xff00
```

设置完数据观察断点, 可以通过点击主菜单 “Break->List” 打开断点列表观察断点的设置情况, 如下图所示。

图五十八、从断点列表观察数据断点的设置情况

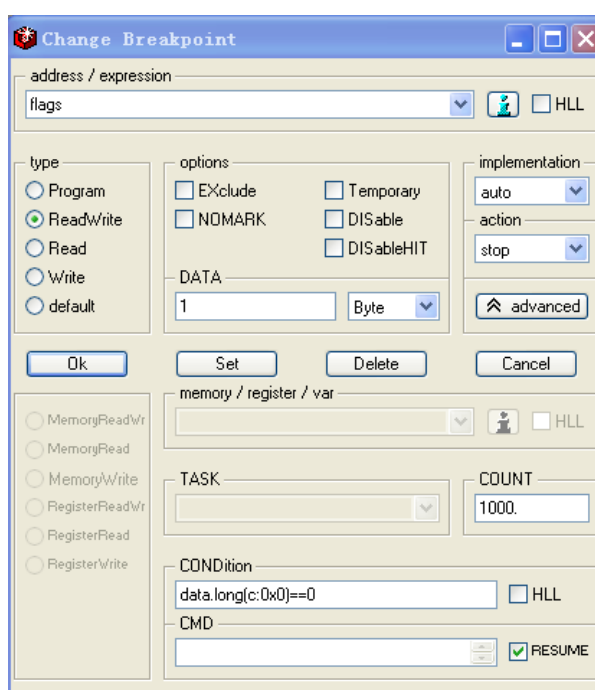


上面的数据断点，用户也可以通过从主菜单点击“Break->Set”打开断点设置对话框，如图五十六所示，在断点设置对话框中设置。

用户也可以在程序列表窗口或存储器观察窗口中点击鼠标右键，选择“Breakpoints...”打开断点设置对话框，在其中设置。

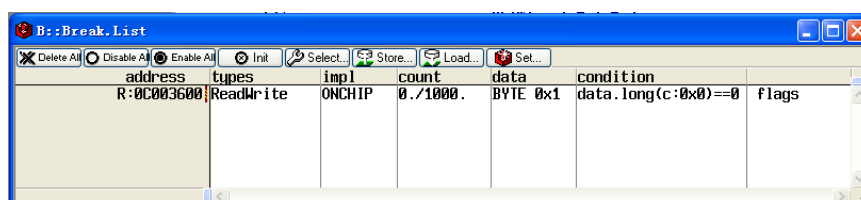
上述数据断点是 Embedded-ICE 直接提供的功能，数量有限，只能设置一个。用户在断点设置对话框中可以点击“advanced”按钮，打开断点设置高级选项对话框，对断点的附属选项（计数、条件、命令）进行设置。如下图所示。

图五十九、带高级选项的数据断点的设置



用户在图五十九中的“count”编辑框中输入计数值，在“condition”编辑框中输入条件，在“CMD”编辑框中输入所要执行的命令。输入完成之后，用户点击“set”按钮设置断点。用户从主菜单点击“Break->List”打开断点列表窗口观察所设的断点。如下图所示。

图六十、带高级选项的数据断点列表

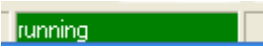


使用高级选项，会影响程序的性能。

3. 13 全速运行程序

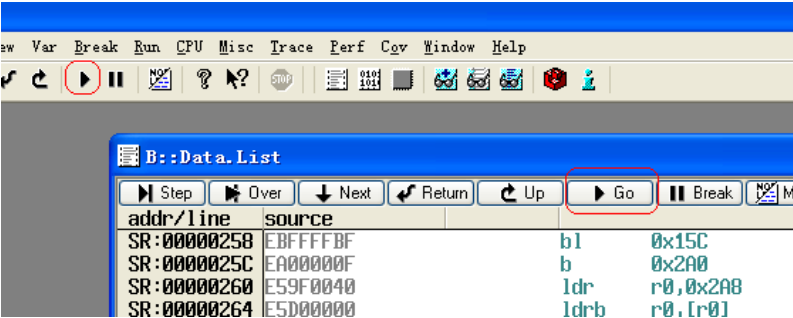
在命令行输入“GO”命令，程序从当前 PC 开始实时全速运行。
要全速运行程序，用户也可以通过主菜单点击“Run->Go”来实现。或者，用户可以按 F7 来全速运行程序。程序全速运行时，在状态显示区会有“Running”指示。如下图所示。

图六十一、全速运行状态指示



用户也可以在主菜单或程序列表窗口点击下图所示的红框中的按钮，也可以实现全速运行程序。

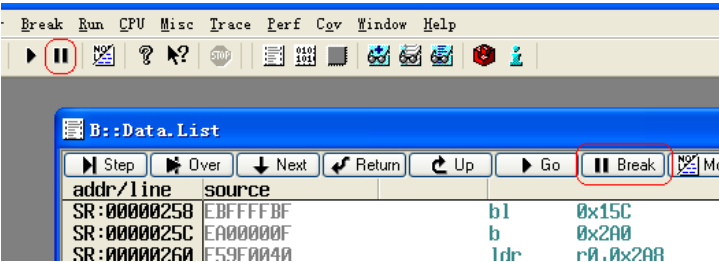
图六十二、全速运行程序的按钮



3. 14 停止运行程序

用户从命令行输入“Break”命令，或者按快捷键 F8，都可以停止运行程序。
用户也可以从主菜单下选择“Run->Break”，停止运行程序。
用户也可以在主菜单或程序列表窗口单击下图中所示红框中的按钮来停止运行程序。

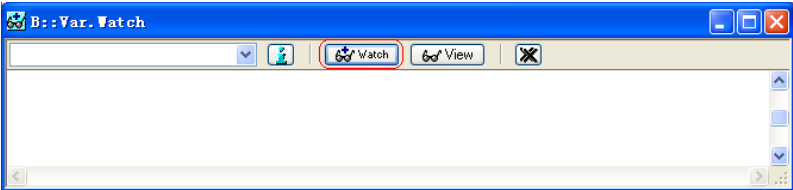
图六十三、停止运行程序的按钮



3. 15 观察变量

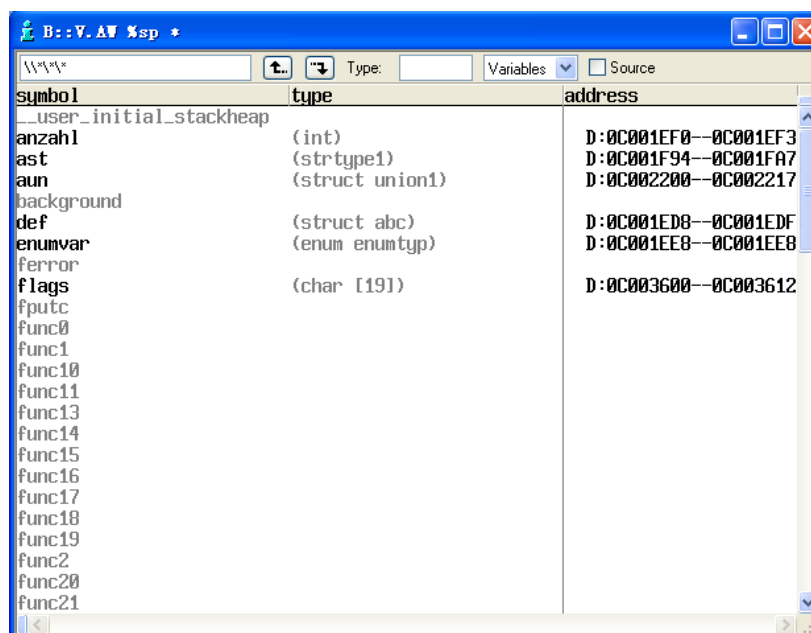
用户通过在命令行输入“Var.Watch”或从主菜单选择“View->Watch”打开变量观察窗口，如下图所示。

图六十四、变量观察窗口

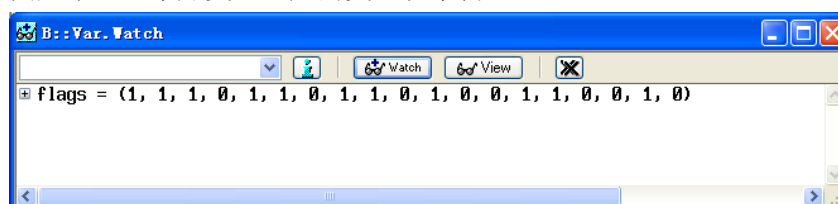


用户点击图六十四中的红框中的“Watch”按钮，打开变量符号列表对话框，如下图所示。

图六十五、变量符号列表对话框

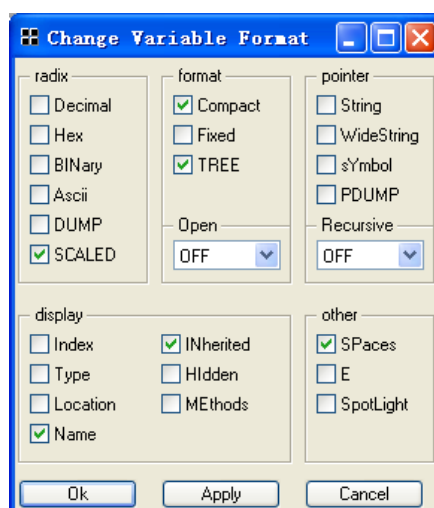


用户在变量符号列表对话框中用鼠标左键双击某一个变量名,把该变量添加到图六十四所示的变量观察窗口中。添加变量之后的变量观察窗口如下图所示。图六十五、添加变量之后的变量观察窗口



在变量观察窗口中的变量名上点击鼠标右键,打开右键辅助功能菜单,从中选择“Format...”,打开变量显示格式设置对话框。如下图所示。

图六十六、变量显示格式设置对话框

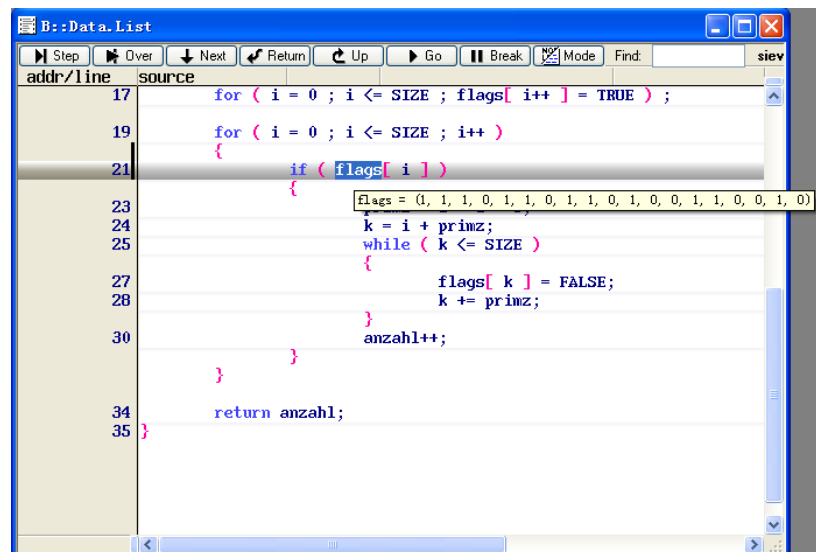


在这个对话框中,用户根据所需要的变量格式进行设置,然后按“Apply”或“Ok”按钮确认。

用户也可以在程序列表窗口中用鼠标右键点击变量名,打开右键辅助功能对话框

框，从中选择“Add to Watch Window”，直接将该变量添加到变量观察窗口中。另外，在程序列表窗口中，用鼠标左键单击变量名时，该变量的值会及时显示出来，如下图所示。

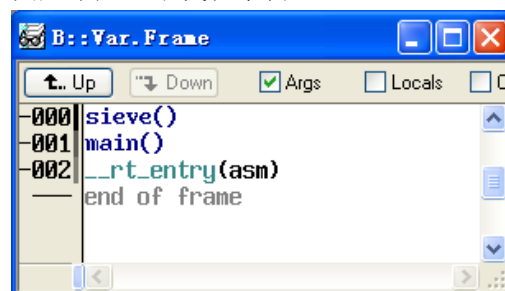
图六十七、显示变量的程序列表窗口



3. 16 观察堆栈

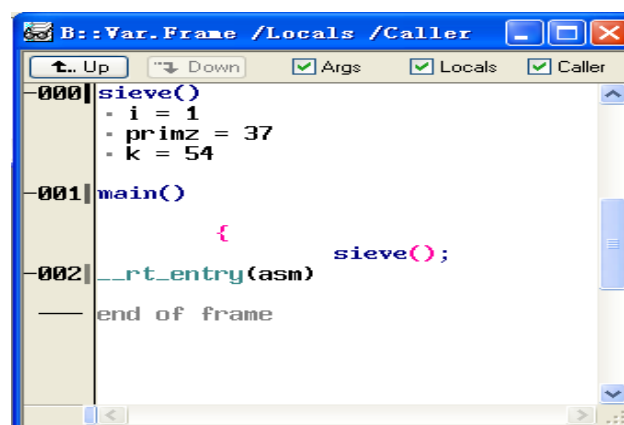
用户在命令行输入“Var.Frame”或者从主菜单选择“View->Stackframe”都可以打开堆栈观察窗口，如下图所示。

图六十八、堆栈观察窗口



用户也可以在命令行输入“Var.Frame /Locals /Caller”或者从主菜单选择“View->Stackframe with Locals”打开带局部变量的堆栈观察窗口，如下图所示。

图六十九、带局部变量的堆栈观察窗口。



3. 17 在线 Flash 编程

用户可以将应用程序直接下载到目标板的片上或片外 Flash 存储器中。一种方法是通过 JTAG 扫描的方式编程,这种方式编程速度慢,一般程序大小在几十千字节的情况下使用。另一种方法是通过一小段运行在目标机上的编程算法程序来完成,这种方法比较通用。这两种算法的区别从命令上体现出来。

这些命令包括:

Flash.Reset – Flash 定义复位,之前所作的所作的所有 Flash 定义全部清除。

Flash.Create – 定义目标板上的 Flash 存储器的类型,位置,总线宽度,扇区大小等信息。

Flash.Target – 只有需要在目标板上运行编程算法的情况中使用。当 Flash.Create 命令中的 Flash 类型是 Target 时,指定需要通过在目标板上运行编程算法。该命令指定编程算法程序、下载到目标板的位置、数据结构的位置、数据区的大小。编程算法程序也可以不用该命令加载,而通过 data.load 命令单独加载。

Flash.Erase – 对所指定的 Flash 进行擦除操作。

Flash.Program – 对所指定的 Flash 进行编程操作。

图七十、通过 JTAG 进行 Flash 编程的例子

```
1 print "Flash erasing..."
2 flash.res
3 flash.c 0--7ffff at49f100 byte
4 flash.program 0--7ffff
5 flash.erase 0--7ffff
6 flash.res
7 print "Flash erased!"
8 wait 1.s
9 print "Flash programming..."
10 flash.res
11 flash.c 1. 0--7ffff at49f010 byte
12 flash.program 0--7ffff
13 data.load.binary * 0x0
14 ;data.load.elf *
15 flash.res
16 print "Flash programming finished!"
17
18
```

图七十所示的例子是把所有的命令都写到了一个批处理脚本文件中的情况,该脚本文件先擦除 Flash,然后再编程。文件中分号后面的内容是注释。Data.load 命令中的星号会打开一个文件打开对话框,用户通过该对话框制定要编程的文件。

图七十一、通过编程算法进行 Flash 编程的例子

```
1 print "Flash erasing..."
2 flash.res
3 flash.c 0--7ffff at49f100 byte
4 flash.program 0--7ffff
5 flash.erase 0--7ffff
6 flash.res
7 print "Flash erased!"
8 wait 1.s
9 print "Flash programming..."
10 flash.res
11 flash.target 0xc000000 0xc004000 0x1000 at49f010.bin
12 flash.c 1. 0--7ffff target byte
13 flash.program 0--7ffff
14 ;data.load.elf *
15 data.load.binary * 0x0
16 flash.res
17 print "Flash programming finished!"
18
19
```